

Vulnerability Hunting

From zero-to-vuln



Prepared for [REDACTED]

About this chapter

- We will learn how to find vulnerabilities
- Manual Vulnerability research
- Fuzzing
- Finding one-days and zero-days

Manual code review or Reverse engineering

- The traditional “old” way of finding vulnerabilities is code-review and reverse engineering
- Always performed with a security-mindset and looking for specific types of bugs.
- Reverse engineering for bug finding is like code review, except you first need to understand the binary → takes longer

Manual code review or Reverse engineering

- For reverse engineering – Use IDA disassembler or similar tool
- For secure code review – use Source Insight
 - Very good for large code bases
 - Great indexing
 - Good for following code flow and data flow in unknown code bases

```

1531: /*
1532:  * split a vma into two pieces at address 'addr', a new vma is allocated either
1533:  * for the first part or the tail.
1534:  */

```

```

1535: int split_vma(struct mm_struct *mm, struct vm_area_struct *vma,
1536:              unsigned long addr, int new_below)
1537: {
1538:     struct vm_area_struct *new;
1539:     struct vm_region *region;
1540:     unsigned long npages;
1541:
1542:     kenter("");
1543:
1544:     /* we're only permitted to split anonymous regions (these should have
1545:      * only a single usage on the region) */
1546:     if (vma->vm_file)
1547:         return -ENOMEM;
1548:
1549:     if (m[]m->map_count >= sysctl_max_map_count)
1550:         return -ENOMEM;
1551:
1552:     region = kmem_cache_alloc(vm_region_jar, GFP_KERNEL);
1553:     if (!region)
1554:         return -ENOMEM;
1555:
1556:     new = kmem_cache_alloc(vm_area_cachep, GFP_KERNEL);
1557:     if (!new) {
1558:         kmem_cache_free(vm_region_jar, region);
1559:         return -ENOMEM;
1560:     }
1561:
1562:     /* most fields are the same, copy all, and then fixup */
1563:     *new = *vma;
1564:     *region = *vma->vm_region;
1565:     new->vm_region = region;
1566:

```

Project Files

Symbol Name

Symbol

```

split_state
split_status_source
split_status_string
split_token_from_
split_transaction_
SPLIT_VIEW
split_vma
split_vma
split_vma
__split_vma
spll
spll
spll
spll
SPLL_CNTL
SPLL_CNTL1

```

Structure in mm_types.h (include/linux) at line 330 (124 lines)

```

struct {
    vm_area_struct * mmap; /* list of VMAs */
    rb_root mm_rb;
    vm_area_struct * mmap_cache; /* last find_vma result */
    FIG_MMU
    ed long (*get_unmapped_area) (struct file *filp,
    unsigned long addr, unsigned long len,

```

Relation References

split_vma::mm

```

split_vma
1. line 1535
2. line 1549
3. line 1592
4. line 1593

```

Benefits of Manual code review

- Straightforward- you know what you are doing
- Helps to understand the code and the attack surface
- Good at finding complex bugs, logical bugs, and bugs that can't be found by other methods

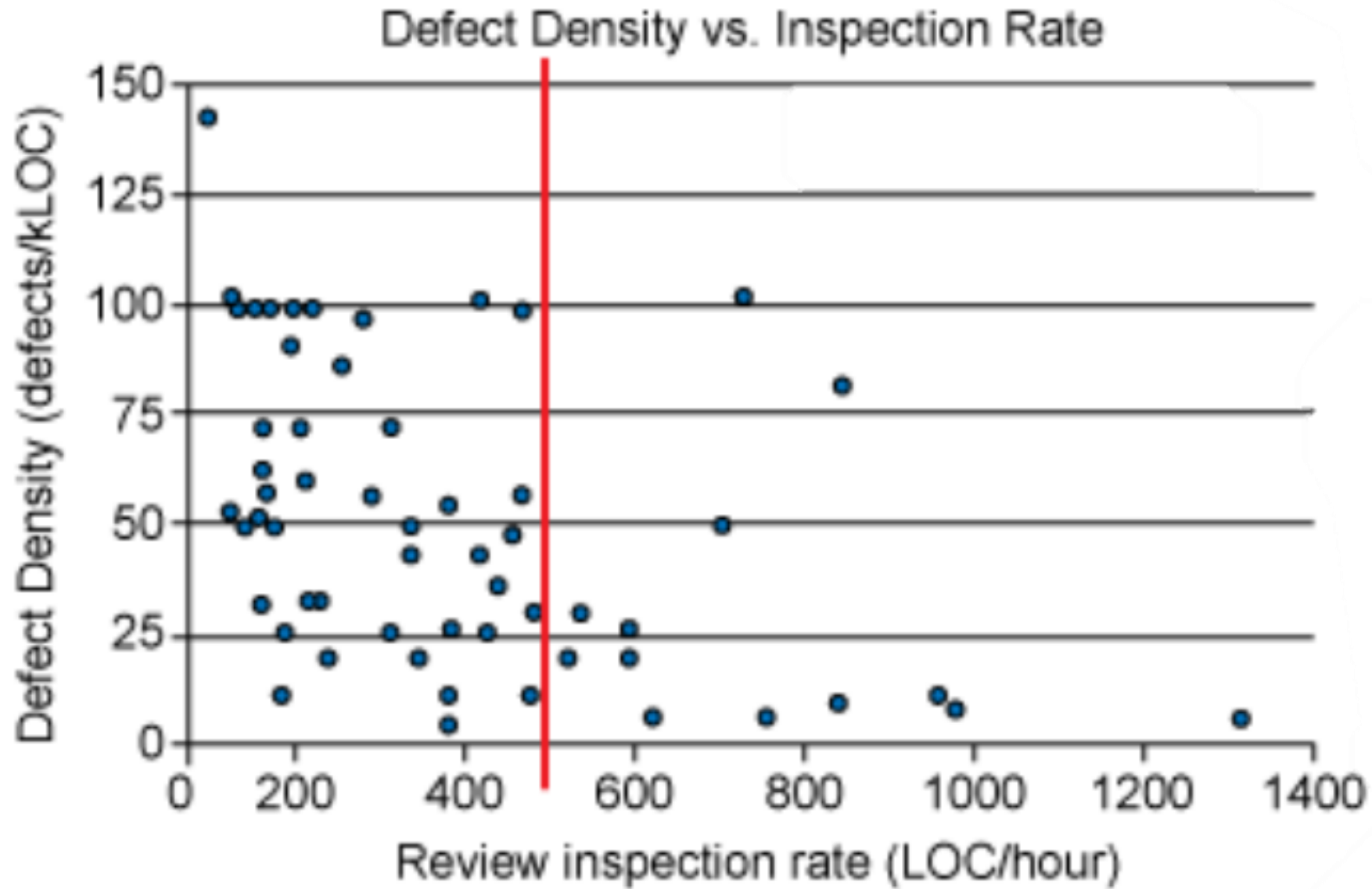
Drawbacks of Manual code review

- Time consuming
- Limited by work hours and skill
- Sisypheic



How to do code review

- Very important to stay focused
- Codebases can be large
- Read few lines of code per hour
 - Guideline for “regular CR” is 400 LOC per hour
 - For security review it will practically be lower
 - Especially for inexperienced reviewer



2-Pass or 3-pass method

- Time-tested approach to secure CR is performing the CR in several passes over the code
- First pass – try to find “low hanging fruit”
 - simple buffer overflow
 - Integer overflow
 - Uninitialized variables
 - Simple logic bugs
 - Missing tests

2-Pass or 3-pass method

- Second pass – understand logic of the program
 - Read as if not a security review
 - Try to understand the program logic
- Third pass - Try to find complex vulnerabilities
 - Race conditions
 - Double-fetch
 - Use-after-free
 - Type confusions
 - Etc..



Code review process

- First get your bearing
- How many LOCs
- What modules
- Is there third-party code?
- Who wrote the code? Are they known for good security
- How mature (==old, maintained) is the code?
- Previous vulnerabilities in the same code-base
 - What types of vulnerabilities do they make?



Code review process

- Prioritize!
- It's impossible to read all the code
- The more effort you put in to a piece of code the more bugs will fall-out
- You wont find all the bugs
 - No, you wont
- Spend more time with code that is likely to have more bugs...

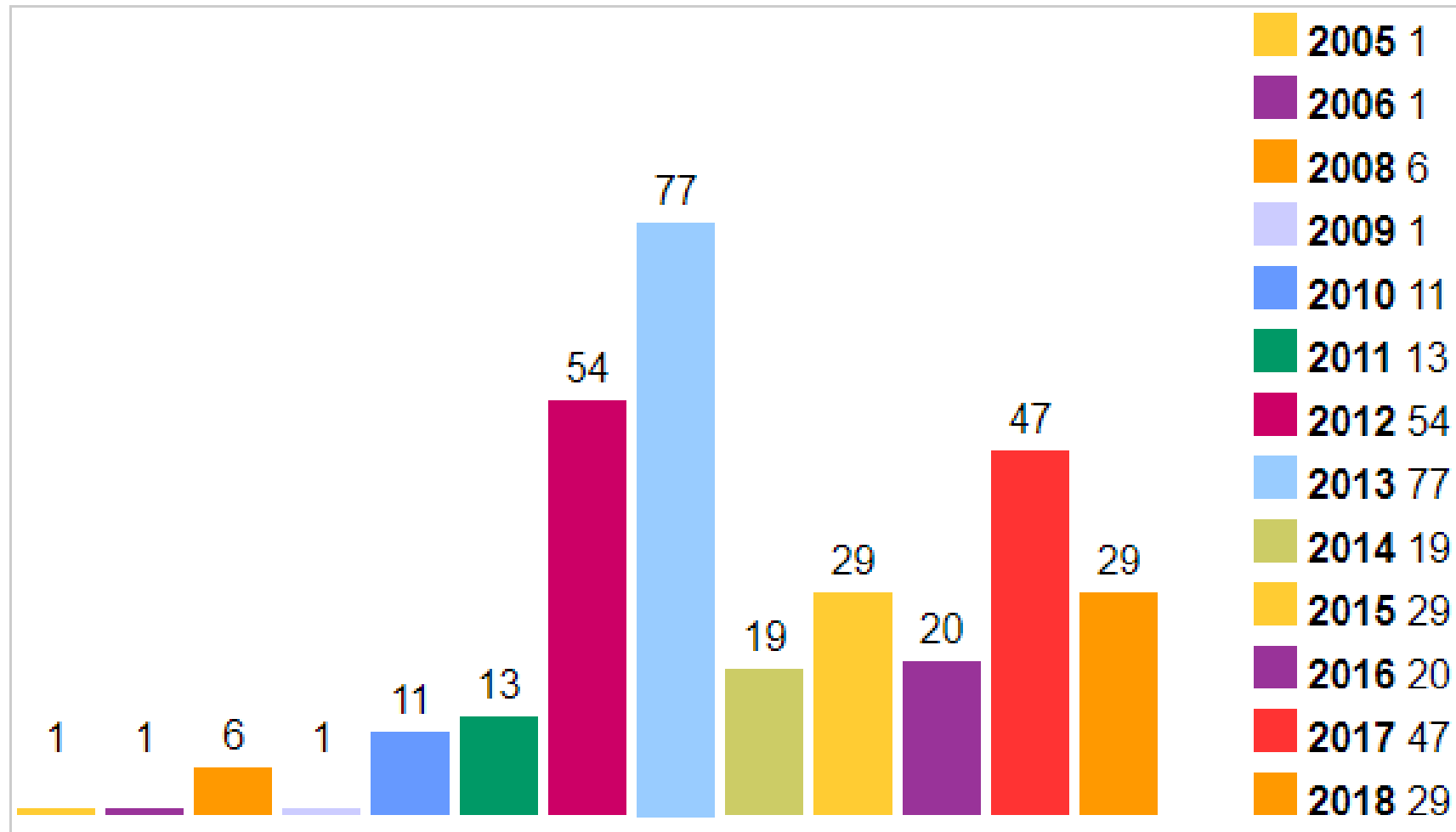
How to prioritize

- Complex looking code
- Code that handles user-input
- Prior bugs
 - Where there were recently bugs there will be more
 - But! They might have a shorter time to closed
- Less mature
- Code Churn!
 - Changing code is vulnerable code

How to prioritize

- Complex code metrics
 - Many levels of nesting
 - Large loops & nested loops
 - Large functions
 - High density of Arithmetic operations, (de)allocations
 - Use of global structs (bonus: in arithmetic operations)
 - unusual, complex, wrong(!) comments
- Certain code constructs:
 - Virtual Machines
 - file-format parsers

FFmpeg vulnerabilities by year



Fuzzing

Finding bugs the automated way



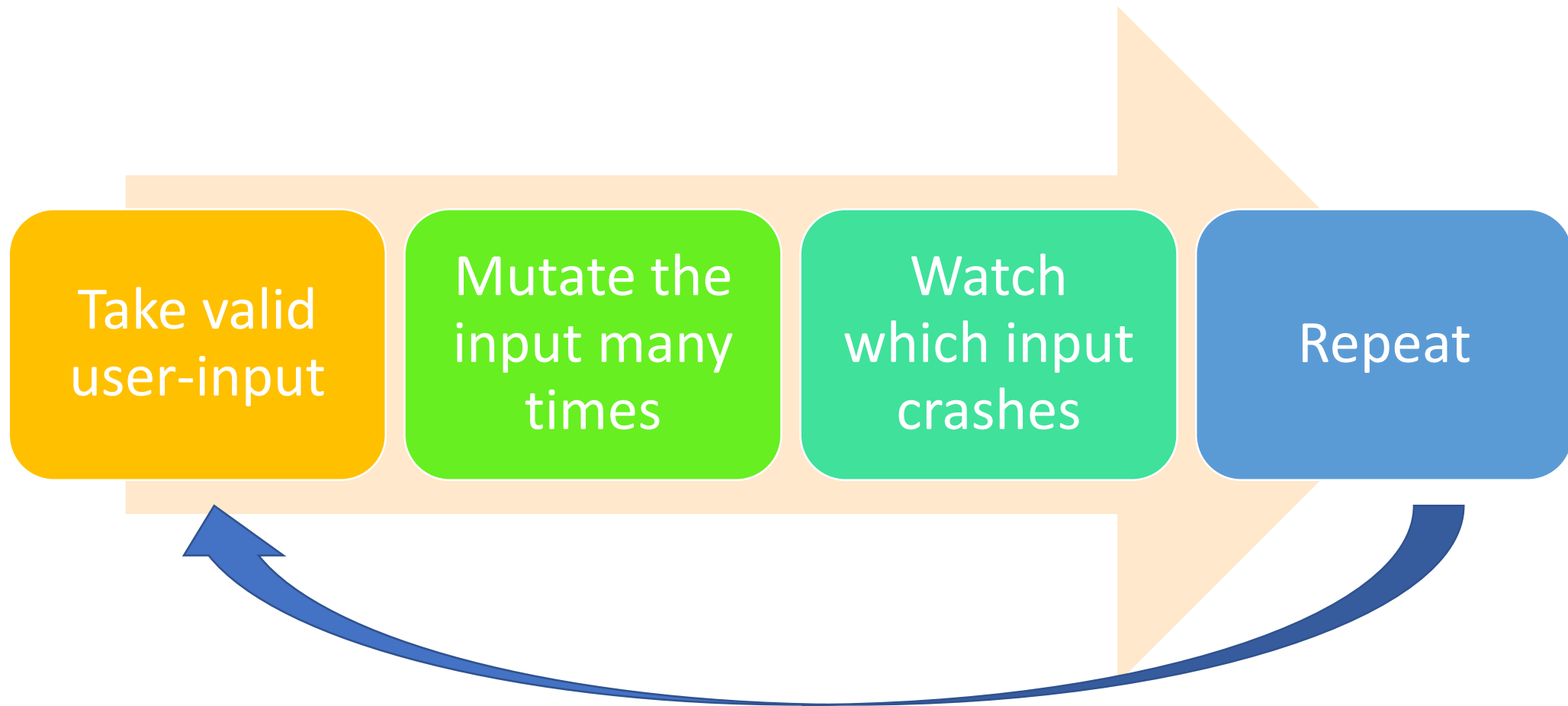
Fuzzing

- Fuzzing is a pretty old method for finding bugs, including security vulnerabilities
- It has made a comeback in recent years with great advances
- Responsible for most of the vulnerabilities found by volume in 2017-2018 (and before)

Fuzzing

- Fuzzing in the most basic term means “throw invalid data at the program”
- See if it crashes
- Do this at volume
- Much to be improved

Fuzzing - The general concept



THE #1 PROGRAMMER EXCUSE
FOR LEGITIMATELY SLACKING OFF:

"MY CODE'S ~~COMPILING~~."

HEY! GET BACK
TO WORK!

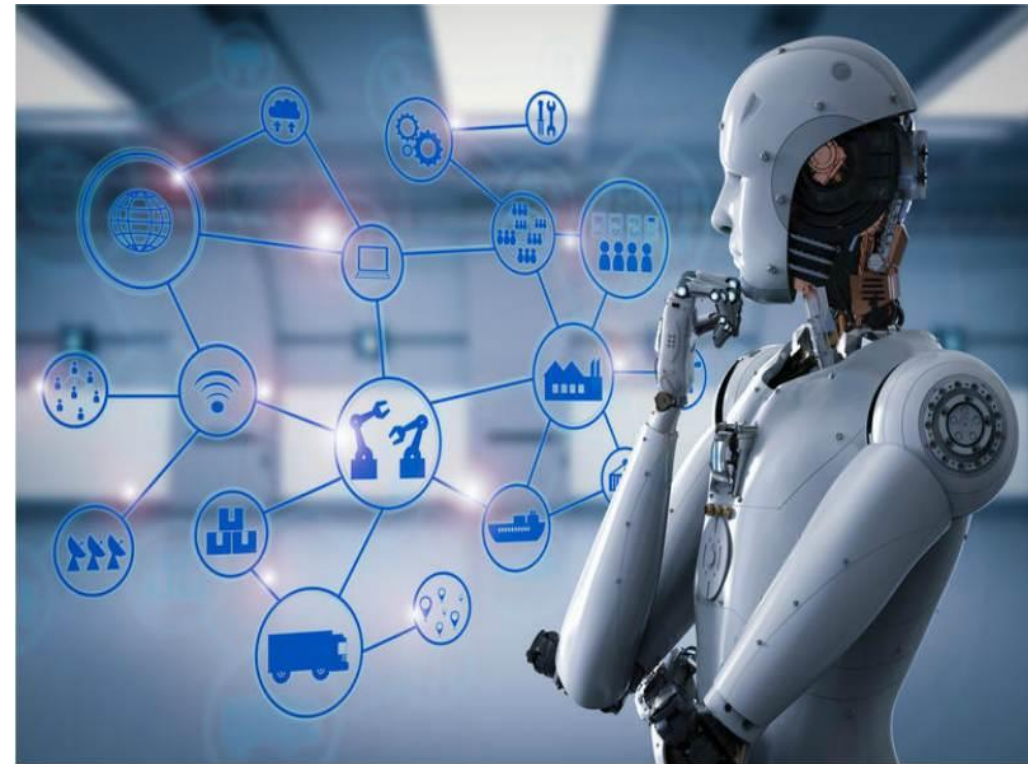
FUZZING!

~~COMPILING!~~

OH. CARRY ON.

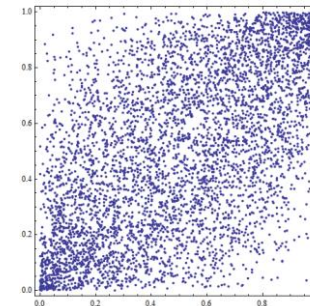
Parts of a Fuzzer

- Mutator
- Test Harness
- Fault detector
- Others:
 - Bug Triage
 - Feedback loop



Fuzzing – What can we improve

- Ok so we have a fuzzer throwing random data at a program
- This used to work a long time ago
- Won't find vulnerabilities today
- Because it's already been done
- This needs to be improved to work



Fuzzing – What can we improve

- Improve mutator algorithm
- Introduce more crashes
 - Some crashes might be hidden
 - ASAN and other sanitizers..
- Perform more iterations per time unit
- Other improvements
 - State-fulness
 - Triggering functionality

Fuzzing – Mutator categories

- “Dumb fuzzing” – bit flipping/other random data modification
- Model-based fuzzing – describe the data format, perform mutations based on the format
- Feedback-based fuzzing - get feedback from the process (usually coverage feedback).. And change fuzzing accordingly → this is the future (and present best practice)

Fuzzing – Dumb fuzzing

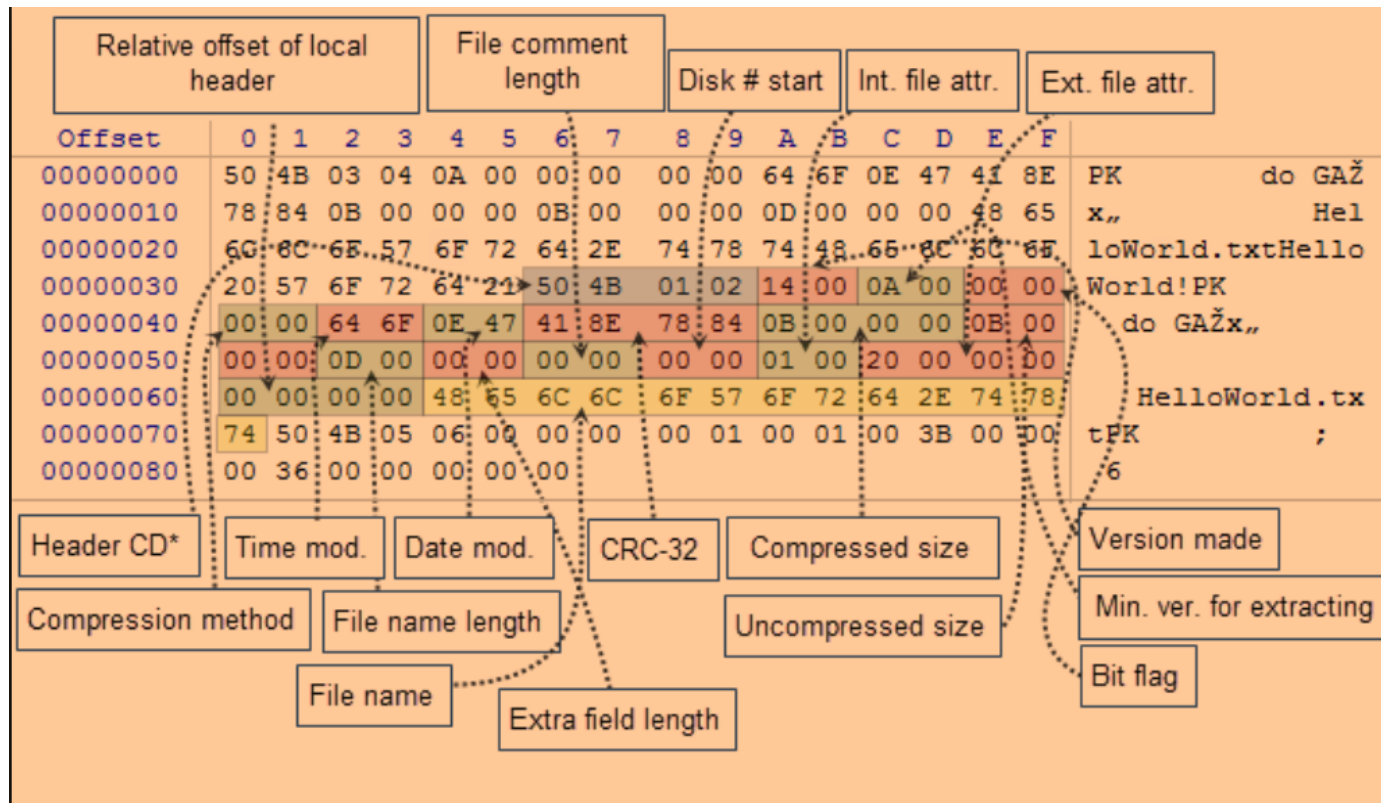
- Flip bits
- Introduce random bytes
- Add or subtract 1 from a value
- Choose edge values (0x00, 0xFFFFFFFF, 1, 0x80000000)
- Combining mixture of multiple inputs

“Charlie miller fuzzer”

```
numwrites = random.randrange(math.ceil((float(len(buf)) / FuzzFactor))) + 1
for j in range(numwrites):
    rbyte = random.randrange(256)
    rn = random.randrange(len(buf))
    buf[rn] = "%c"%(rbyte)
```

Fuzzing – Model Based Fuzzing

- Some markup language is used to define the “model”
- For example: first 4 bytes are a magic value, next two bytes are the length of a structure
- The fuzzing is then performed in an aware manner
- Allows more sophisticated fuzzing
 - Duplicate this element
 - Make the length 1-off from the real length
 - Decompress data, mutate, compress



• DataModel in Peach Pit

```

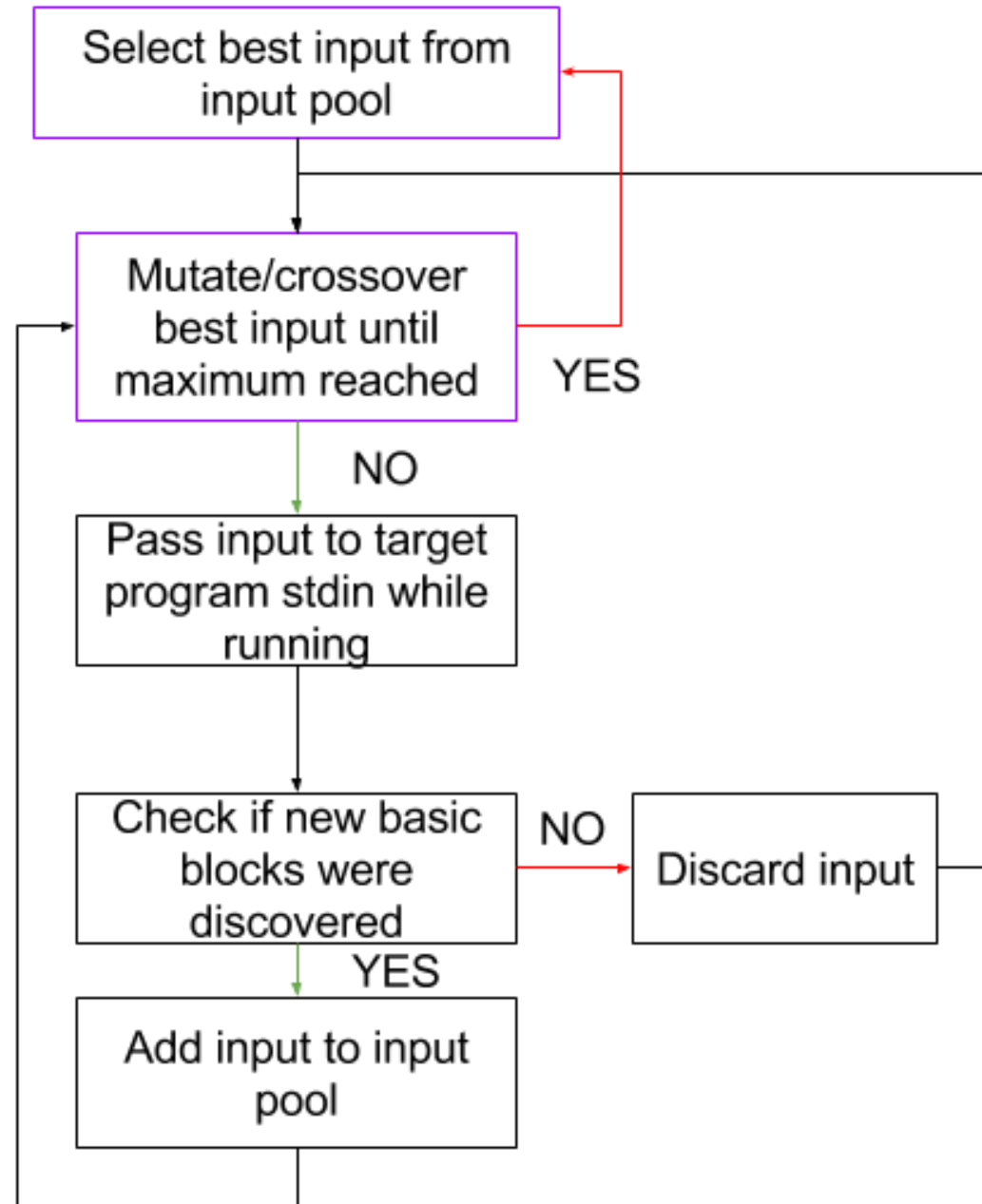
<DataModel name="PNGTest">
  <Blob name="head" length="8" mutable="false"/>
  <Number name="dataLength" size="32" valueType="hex" endian="big">
    <Relation type="size" of="chunk" expressionGet="size+4" expressionSet="size-4"/>
  </Number>
  <Block name="chunk">
    <String name="chunkHead" length="4">
      <Hint name="ValidValues" value="pHYs;fcTL;IDAT;"/>
    </String>
    <Number name="width" size="32" valueType="hex" endian="big"/>
    <Number name="height" size="32" valueType="hex" endian="big"/>
    <Number name="enum" size="8" minOccurs="1"/>
  </Block>
  <Number name="CRC" size="32" endian="big">
    <Fixup class="Crc">
      <Param name="ref" value="chunk"/>
    </Fixup>
  </Number>
</DataModel>

```

Address	Hex	ASCII
0000h	89 50 4E 47 0D 0A 1A 0A	PNPNG...
0010h	00 00 00 00 00 00 00 01 03 00 00 00 6D CC 6B	IDAT...
0020h	C4	

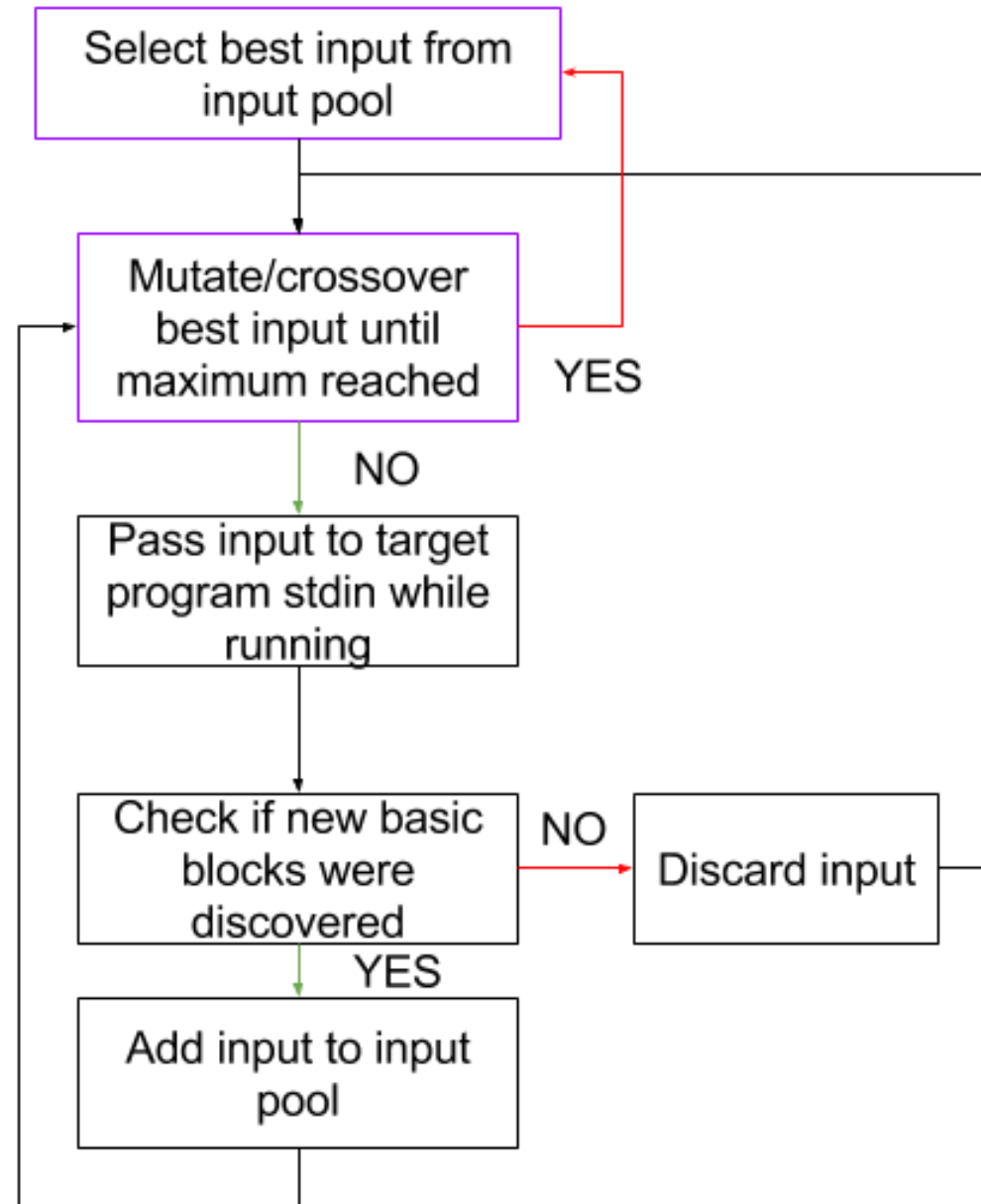
Fuzzing – Feedback Based Fuzzing

- This is the current state-of-the-art
- Feedback based fuzzing gathers feedback of which code areas were run for every test case
- Prioritize and mutate to optimize for max code coverage/time
 - Or some approximation
- HUGE advancement in fuzzing techniques

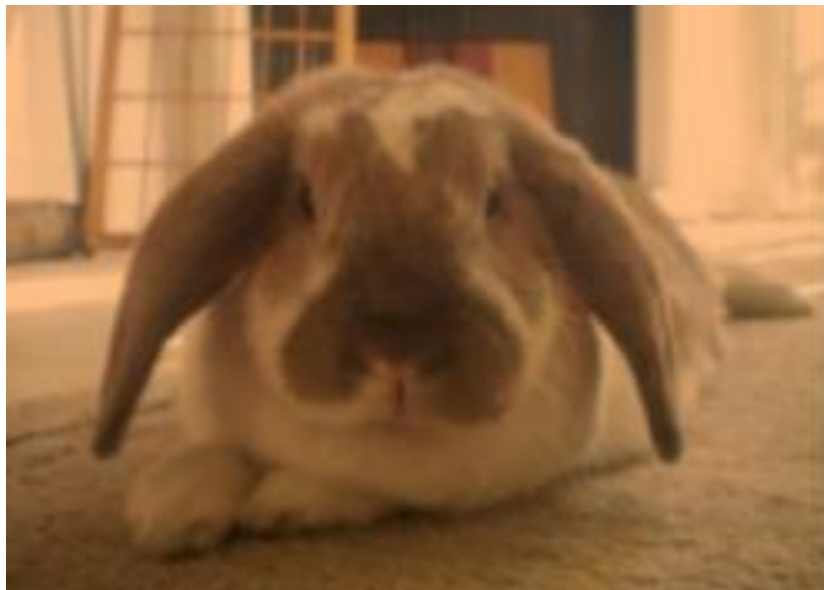


Fuzzing – Feedback Based Fuzzing

- By maximizing code coverage we exercise more functionality of the program
- This leads to more bugs
- “Corpus distillation” – build a minimal corpus of inputs to exercise maximal program states

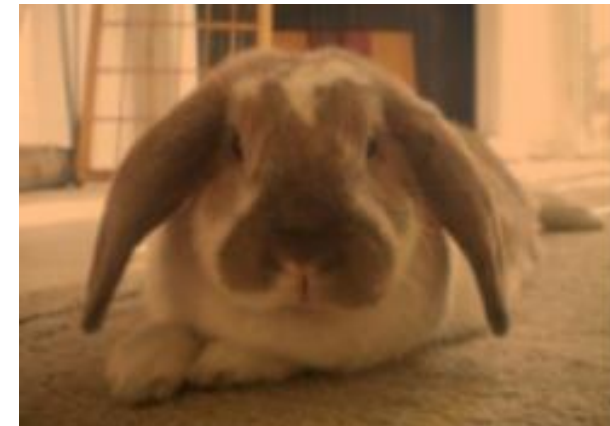


Fuzzing - AFL



Fuzzing – AFL

- One of the most popular Fuzzers for file formats
- Written by “Michał (lcamtuf) Zalewski” formerly from google and a prominent security researcher
- Very effective Fuzzer
- Short for “American Fuzzy Lop”



Fuzzing – AFL

- Feedback based – maximizes for edge coverage
- Very Fast
- Very robust
- Works
 - **Thousands of vulnerabilities found**
- Low setup time
 - If not trying to do anything fancy



Fuzzing – AFL

- Works by adding instrumentation in the compilation stage
 - Special version of GCC or CLANG

```
CC=/path/to/afl-gcc CXX=/path/to/afl-g++ ./configure --disable-shared  
make clean all
```

- Fast, efficient instrumentation measuring Basic Block Edges covered

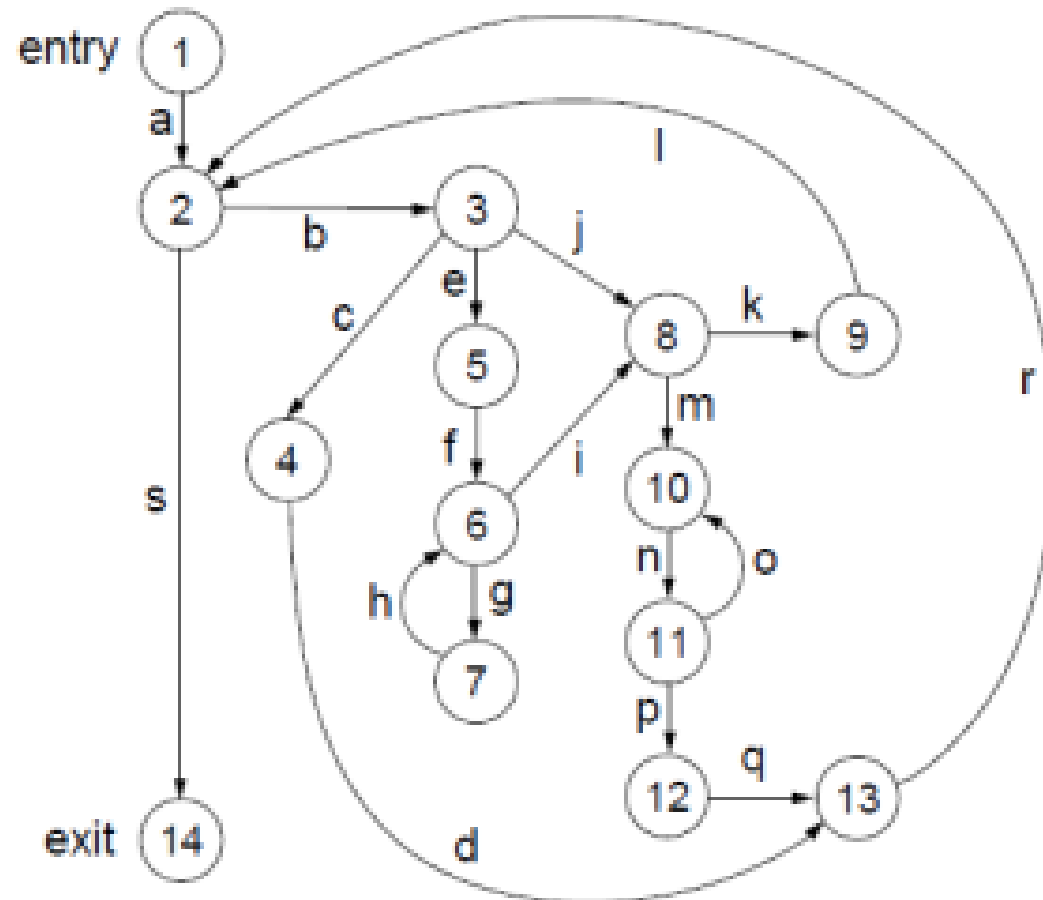
```
cur_location = <COMPILE_TIME_RANDOM>;  
shared_mem[cur_location ^ prev_location]++;  
prev_location = cur_location >> 1;
```

Fuzzing – AFL

```

e1;
while (e2) {
  switch (e3) {
    case 1: e4;
             break;
    case 2: e5;
             while (e6) e7;
    default:
      if (e8) {
        e9;
        continue;
      }
      do e10; while (e11);
      e12;
  }
  e13;
}
e14;

```



Fuzzing – AFL

- Works by adding instrumentation in the compilation stage
 - Special version of GCC or CLANG

```
CC=/path/to/afl-gcc CXX=/path/to/afl-g++ ./configure --disable-shared  
make clean all
```

- Fast, efficient instrumentation measuring Basic Block Edges covered

```
cur_location = <COMPILE_TIME_RANDOM>;  
shared_mem[cur_location ^ prev_location]++;  
prev_location = cur_location >> 1;
```


Fuzzing – AFL

- Skips program initialization with a “fork server”
 - Fork() after the initialization
 - Change cmdline parameters
- Provides 2X improvement
- With some help, can provide 10X improvement or more

Fuzzing – AFL

- AFL can support a client-server setup to massively parallelize fuzzing
 - Between computers
- Several variant instrumentation methods
 - GCC
 - Clang
 - Clang-fast
- Clang instrumentation is better – since the compiler can optimize it

Fuzzing – AFL

- Different types of mutators
- Genetic and “dumb” mutation
- Can change mutators
- Selects “promising” candidates for future fuzzing
 - Candidates that got new paths recently
 - have potential of getting new paths

american fuzzy lop 0.47b (readpng)

process timing

run time : 0 days, 0 hrs, 4 min, 43 sec
 last new path : 0 days, 0 hrs, 0 min, 26 sec
 last uniq crash : none seen yet
 last uniq hang : 0 days, 0 hrs, 1 min, 51 sec

cycle progress

now processing : 38 (19.49%)
 paths timed out : 0 (0.00%)

stage progress

now trying : interest 32/8
 stage execs : 0/9990 (0.00%)
 total execs : 654k
 exec speed : 2306/sec

fuzzing strategy yields

bit flips : 88/14.4k, 6/14.4k, 6/14.4k
 byte flips : 0/1804, 0/1786, 1/1750
 arithmetics : 31/126k, 3/45.6k, 1/17.8k
 known ints : 1/15.8k, 4/65.8k, 6/78.2k
 havoc : 34/254k, 0/0
 trim : 2876 B/931 (61.45% gain)

overall results

cycles done : 0
 total paths : 195
 uniq crashes : 0
 uniq hangs : 1

map coverage

map density : 1217 (7.43%)
 count coverage : 2.55 bits/tuple

findings in depth

favorable paths : 128 (65.64%)
 new edges on : 85 (43.59%)
 total crashes : 0 (0 unique)
 total hangs : 1 (1 unique)

path geometry

levels : 3
 pending : 178
 pend fav : 114
 imported : 0
 variable : 0
 latent : 0

Fuzzing – AFL - Other Features

- Dedicated allocator to catch more bugs
- Tools to try to reconstruct data model
- Tool to minimize corpus
- Tool to minimize a single file

Fuzzing – AFL - Other Features

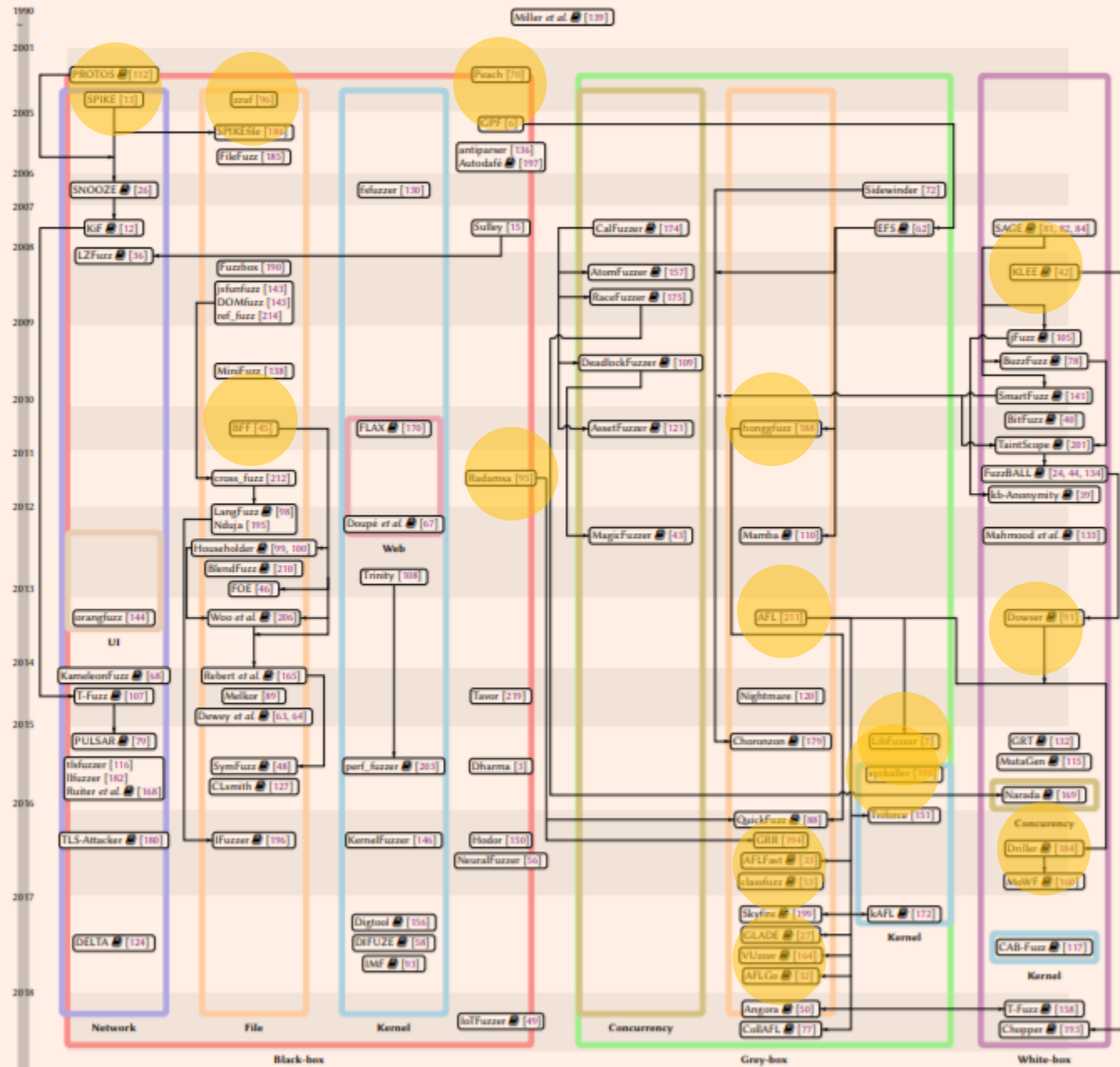
- QEMU mode for closed-source
- WinAFL version for closed-source on windows
 - Actually a derivative but worth note
- Tool to deduce program grammar on it's own..
 - Sort of

Fuzzing – AFL - Derivatives

- Many people have tried to improve/derive work from AFL
- It's like a (inter-)national sport
- Many specific improvements for different needs
 - Including fuzzing binary closed-source targets
- Different instrumentation types for closed source
 - PIN, DynamoRIO, tracing..

Other notable fuzzers

- Peach – famous model-based fuzzing
- Syzkaller, honggfuzz, libfuzzer – other google feedback-based fuzzers
- Codenomicon – commercial model-based fuzzing
- Radamsa – older general-purpose “dumb” fuzzer with many useful mutations
- Domato – DOM-parse fuzzer
- Zzuf - “inline fuzzer”
- Vuzzer- fairly new taint analysis + guided fuzzer
- Angora – fairly new taint analysis + guided fuzzer



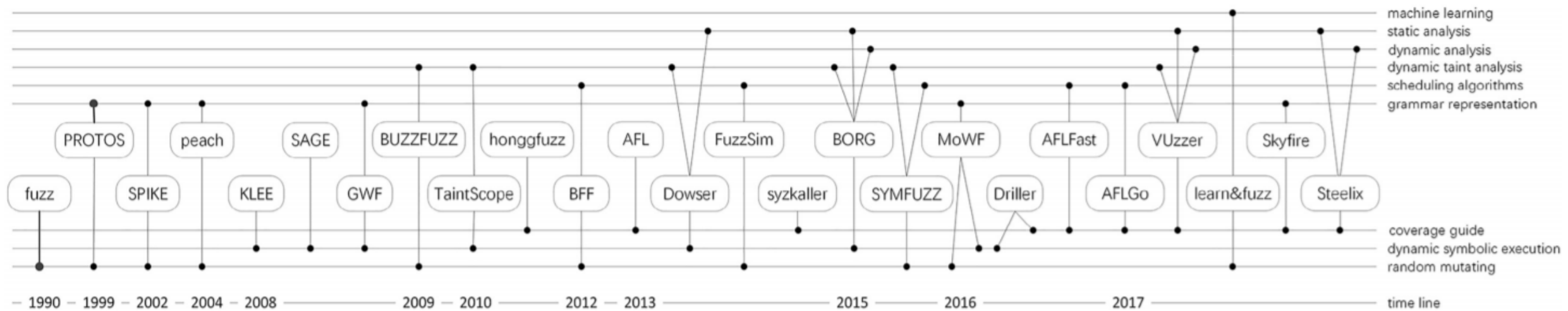


Fig. 3 – Fuzzing Technique Evolution Diagram.

Circumventing Fuzzing Roadblocks with Compiler Transformations

Posted on August 15, 2016 by lafintel

TL;DR: We build some LLVM passes which 'deoptimize' code generated by LLVM to increase code coverage with AFL (and potentially other feedback driven fuzzers, e.g. libFuzzer). Get the code [here](#).

```
if (input == 0xabad1dea) {  
    /* terribly buggy code */  
} else {  
    /* secure code */  
}
```



```
if (input >> 24 == 0xab){  
    if ((input & 0xff0000) >> 16 == 0xad) {  
        if ((input & 0xff00) >> 8 == 0x1d) {  
            if ((input & 0xff) == 0xea) {  
                /* terrible code */  
                goto end;  
            }  
        }  
    }  
}  
  
/* good code */  
  
end:
```

Other methods to aid fuzzing

- Symbolic Execution
- Taint analysis
- Others and combinations
- These methods have a stronger academic background

Symbolic Execution

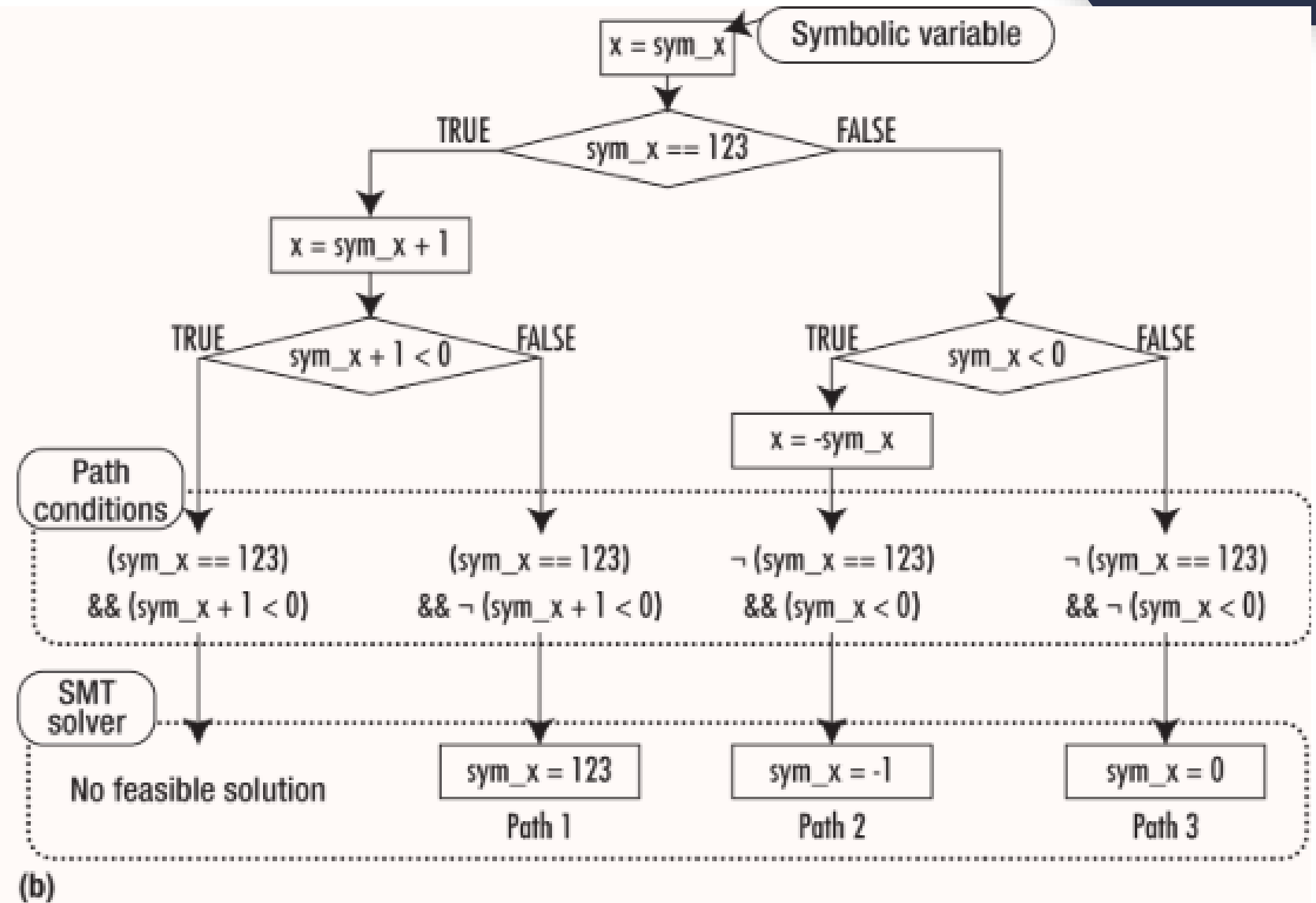
- Take a symbol
- Mark the input as a “symbolic variables” – an algebraic variable
- Define the limitations for code to get to a certain path as a “constraint”
- Try to solve the constraint → Get to a path

```

1 int buggy_abs(int x) {
2   if (x == 123)
3     x++;
4   if (x < 0)
5     x = -x;
6   return x;
7 }

```

(a)



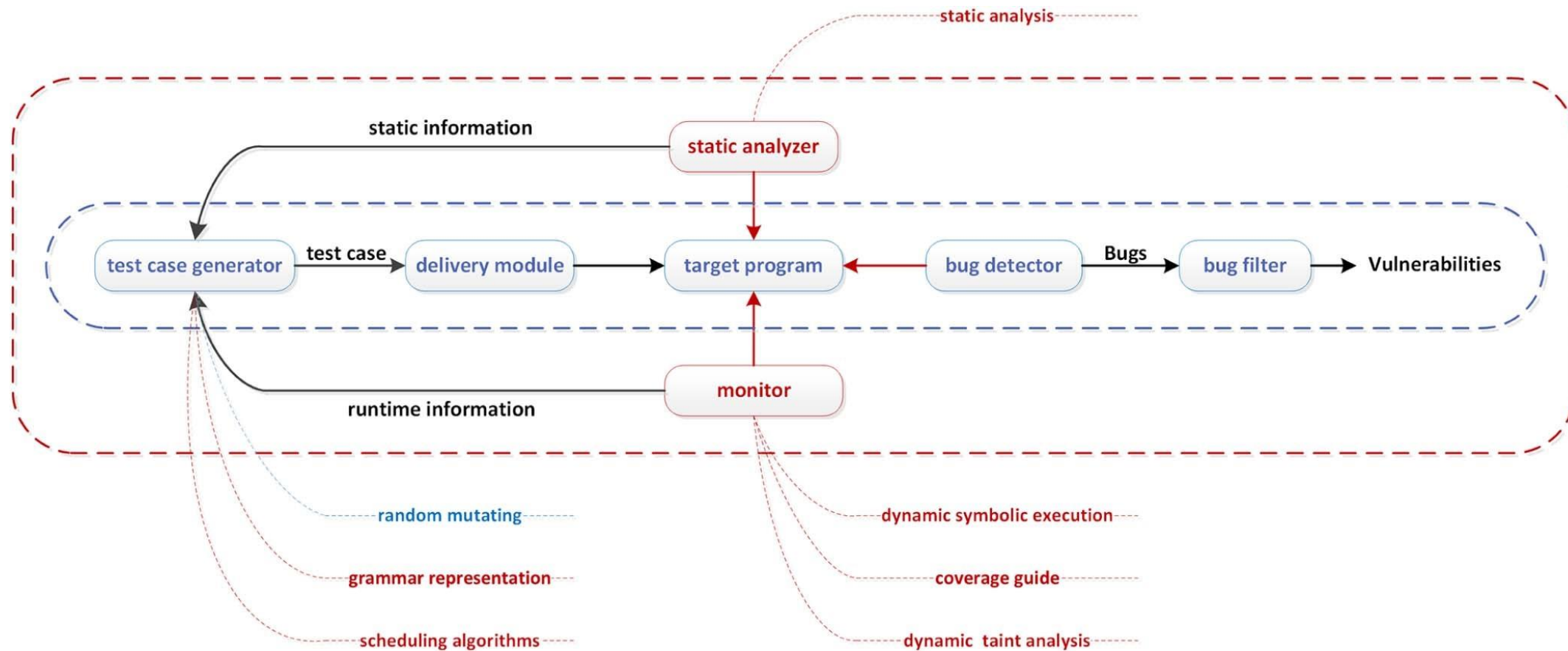
(b)

Symbolic Execution

- Major limitation in symbolic execution is “path explosion”
- Too many path leads to complex constraints
- Loops quickly expand the amount of paths
- Usually this is solved with all kinds of hybrid solution and limiting-constraints

Notable Advanced Fuzzers and SymEx

- KLEE
- ANGR
- DRILLER (Hybrid)
- VUZZER (Hybrid)
- STEELIX (Hybrid)



Comparison of methods Fuzzing Vs. Code Review

- Fuzzing is more automated
- Fuzzing gives faster results on “unexplored” targets
- Fuzzing always produces the same results so if target was already fuzzed with the same Fuzzing method...
- Fuzzing sometimes gives a false sense of understanding
- Fuzzing will seldom detect complex logic bugs
 - and will only detect bugs if the fault can be detected..

Fuzzing tips

- Always make sure you have coverage
 - Even if not using a coverage-based fuzzer
- Test with an older version of the same library or tool to detect known bugs first
 - Or insert your own bug and find it
 - This is a “sanity check”
- Don't expect miracles

Fuzzing tips

- Don't expect it to be a “quick setup” and leave to run
- Put effort in to learning the format, optimizing all parts of the fuzzer for the target
 - Fast test harness
 - Sanitizers
 - Good corpus of input samples
 - Mutators
 - Compute-power
 - Human adaptations whenever coverage becomes flat
- **Always be improving coverage**

When to use fuzzing

- If your target might be “unexplored”
- When there is lots of (complex) parsing logic

Or

- If you have enough capacity to do this in parallel to CR
- If you have enough capacity to make novel contributions to the Fuzzer

